Multi-Channel Taskers

Amos Hebb
Department of Electrical and Computer Engineering
University of Toronto
Toronto, Canada
a.hebb@mail.utoronto.ca

Sunila Akbar
Department of Electrical and Computer Engineering
University of Toronto
Toronto, Canada
sunila.akbar@utoronto.ca

Raviraj S. Adve
Department of Electrical and Computer Engineering
University of Toronto
Toronto, Canada
rsadve@ece.utoronto.ca

Zhen Ding
Radar Sensing and Exploitation Section
Defence Research and Development Canada (DRDC)
Ottawa, Canada
jack.ding@ecn.forces.gc.ca

Peter W. Moo
Radar Sensing and Exploitation Section
Defence Research and Development Canada (DRDC)
Ottawa, Canada
peter.moo@ecn.forces.gc.ca

***Abstract*—We develop a more comprehensive simulation of a suite of sensors to evaluate radar resource management scheduling algorithms.**

***Index terms*—Cognitive Radar, Multifunction Radar, Radar Resource Management**

I. Introduction

Q-Ram *Q-Ram*

II. Background

Text [1] and [2] .

A. What does a boat look like?

The main goal is a more comprehensive simulation of a suite of sensors. Decisions made in this work are inspired by the *River-class destroyer*.

- Aegis Combat System with Canadian Tactical Interface.
- USN Cooperative Engagement Capability (sensor netting).
- Integrated Cyber Defense System.
- OSI Maritime Systems Integrated Bridge and Navigation System.
- L3Harris Internal and External Communication Suite.
- Lockheed Martin AN/SPY-7(V)3 Solid State 3D AESA radar
- MDA Solid State AESA Target Illuminator
- X Band "Surface search antenna" navigation radar.
- S Band navigation radar.
- L3Harris WESCAM Electro-optical and infrared systems.
- NA-30S MK-2 Fire Control System - deal band X-band for search & acquisition, Ka-band for DART guided ammunition.

We do not model underwater sensors, electronic warfare, aviation facilities, or armament systems.

a) *Aegis Combat System:*

Includes an "Aegis Common Source Library" (CSL) and "Open Architecture". Appears to be closed source.

It manages the detect-to-engage sequuence for anti-air warfare. The AESA SPY-7 radar autonomously detects and tracks contacts. Other components classify and identify system tracks.

b) *SPY-7:*

- 1.64 times the detection range of SPY-1
- S-Band radar
- Long range *discrimination* radar.
- Software wizardry to reduce the resolution difference between it and a dedicated X-band radar.

c) *Radar Bands:*

- L-Band 1-2GHz
- S-Band 2-4GHz
- C-Band 4-8GHz
- X-Band 8-12GHz

d) *Functions:*

- Search
- Confirmation
- Tracking
- Cued search

e) *Clutter:*

- Land
- Sea
- Chaff
- Rain
- Angels (flocks of birds or insects)
- Jammers?

III. System Model

The previous work by Akbar *et al.* [2] did not consider the full system model, but instead model the distributions of task parameters. We extend this work by considering the full system model, including the radar and the environment. This is necessary because the radar and the environment are not independent of the task parameters. By considering the full system model, we get a model of the impact of scheduling decisions on the task parameters for free.

A. Radar Model

Surveillance Region is an annulur sector with a radar at the origin, an inner radius of 10 km, an azimuth $\theta \in [-45°, 45°]$ and range $r \in [10\ \text{km}, 184\ \text{km}]$.

IV. Implementation

In earlier attempts to model radar scheduling problems [3], accelerators and distributed resources[1] were poorly utilized due to the environment being a bottleneck. We address this by using JAX [4] to write both our environment model and our agent. This allow us to just-in-time compile (`jit`) the entire training loop using a single-program multiple-data pattern. Writing within the constraints of JAX gives us the benefit of easily running thousands of seeds in thousands of environments in parallel on a single GPU/TPU, or across multiple GPUs with parallel mapping.

A. Compatability

There is no way to build a complex simulator that produces structured data and get that data into a standard learning algorithm. PufferLib [5] takes the data and packs it into a flat buffer, this allows a complex simulator to be interacted with like an Atari game.

GPU acceleration for simulation. Just write for CPU and use PufferLib's emulation layer to run in parallel.

Custom simulation. Pure C with zero dynamic memory allocations. c_moba.pyx Train at over 1,000,000 steps per second. Centuries of simulated data in a few hours.

V. Simulation Results

Performance metrics are presented in three areas: scheduling, detection, and tracking.

A. Scheduling

B. Detection

C. Tracking

VI. Final Remarks

[1]This research was enabled in part by staff expertise and computing provided by the Digital Research Alliance of Canada (https://alliancecan.ca)

References

[1] S. Akbar, R. S. Adve, Z. Ding, and P. W. Moo, "Transfer-based DRL for task scheduling in dynamic environments for cognitive radar," *IEEE Transactions on Aerospace and Electronic Systems*, 2023.

[2] S. Akbar, R. S. Adve, Z. Ding, and P. W. Moo, "Model-Based DRL for Task Scheduling in Dynamic Environments for Cognitive Multifunction Radar," in *2024 IEEE Radar Conference (RadarConf24)*, 2024.

[3] A. Hebb, R. S. Adve, Z. Qu, and Z. Ding, "Belief-Rewards to Learn Radar Scheduling Policies," in *2024 IEEE Radar Conference (RadarConf24)*, 2024, pp. 1–6. doi: 10.1109/RadarConf2458775.2024.10548136.

[4] R. Frostig, M. J. Johnson, and C. Leary, "Compiling machine learning programs via high-level tracing," *Systems for Machine Learning*, vol. 4, no. 9, 2018.

[5] J. Suárez, "PufferLib: Making Reinforcement Learning Libraries and Environments Play Nice." [Online]. Available: https://arxiv.org/abs/2406.12905